

INTRODUCTION

- **Dictionary** are mutable, unordered collection with elements in the form of a **key:value** pairs that associate keys to values.
- Rather than index associated with lists, tuples and strings, a dictionary has key associated with values.
- It is just like English Dictionary where meaning is associated with a word (Key).
- Dictionaries are containers that associate keys to values.
- With list, tuples and strings we need to know the index of individual element to access but with Dictionaries user need to search value based on key.

CREATING AND ACCESSING LISTS

```
>>>Month={"January":31,"February":28,"March":31,"April":30,"May":31,"June":30,"July":31,"August":31,"September":30,"October":31,"November":30,"December":31}
```

```
>>> teacher={'Bimlendu':'9470010804','Prakash':'8604774180'}
```

Notice that:

1. Curly braces mark the beginning and end of the dictionary
2. Each entry (key:value) consists of a pair separated by a colon. The key and corresponding value is given by writing colon(:) between them.
3. The key:value pairs are separated by comma.

Internally dictionaries are indexed (i.e.) arranged on the basis of keys.

SOME MORE DICTIONARY DECLARATION

Emptydictionary={ }

```
>>>daynumberofweek={"Monday":1, "Tuesday":2, "Wednesday":3,  
    "Thursday":4,"Friday":5,"Saturday":6,"Sunday":7}
```

```
>>> subjectandcode={"Physics":42,"Chemistry":43,  
    "Mathematics":41, "Biology":44,"Computer  
    Science":83,"Informatics Practices":65,"English":101,"Hindi":2}
```

Note:

- Dictionaries are also called **Associative Arrays** or **mappings** or **hashes**.
- **Keys of a dictionaries must be of immutable type** such as Python string, Number, a tuple (containing only immutable entry) but list which is mutable can not be used as keys of a dictionary.

```
>>>dict2={ [2,3]: "abc" }      #TypeError: Un-sharable Type 'list'
```


ACCESSING OF A DICTIONARY

Its general syntax is
dictionaryname<"key">

EXAMPLE:

```
>>> subjectandcode["Hindi"]
```

```
2
```

```
>>> subjectandcode["Computer Science"]
```

```
83
```

```
>>> subjectandcode["Informatics Practices"]
```

```
65
```

ACCESSING ENTIRE DICTIONARY

>>>dictionaryname

- Mentioning only the dictionary name without any key prints the entire dictionary.
- If the key in square bracket is used along with dictionaryname produces the value that matches the key otherwise error is displayed.

Example:

```
>>> subjectandcode
```

```
{'Physics': 42, 'Chemistry': 43, 'Mathematics': 41, 'Biology': 44, 'Computer  
Science': 83, 'Informatics Practices': 65, 'English': 101, 'Hindi': 2}
```

```
>>> subjectandcode["Hindi"]
```

```
2
```

ACCESSING ENTIRE DICTIONARY

- A dictionary operation that takes a key and finds the corresponding value is called **lookup**.
- To access a particular value of the dictionary the key is provided in square bracket is in double quote i.e. of string type.
- In python the elements (key:value pairs) are unordered. It means one can not access elements as per specific order.
- References of keys and values are stored in dictionaries.
- Dictionaries are **unordered set of elements**, the printed order of elements may or may not be in the order in which we have stored them in the dictionary.

Access Dictionary Keys: Value one by one through Loop

```
>>> for subject in subjectandcode:  
    print(subject, ":", subjectandcode[subject])
```

Physics : 42

Chemistry : 43

Mathematics : 41

Biology : 44

Computer Science : 83

Informatics Practices : 65

English : 101

Hindi : 2

Accessing Keys and Values

```
>>> subjectandcode.keys()
```

```
dict_keys(['Physics', 'Chemistry', 'Mathematics', 'Biology', 'Computer  
Science', 'Informatics Practices', 'English', 'Hindi'])
```

```
>>> subjectandcode.values()
```

```
dict_values([42, 43, 41, 44, 83, 65, 101, 2])
```

It can be converted in list as below

```
>>> list(subjectandcode.keys())
```

```
['Physics', 'Chemistry', 'Mathematics', 'Biology', 'Computer Science',  
'Informatics Practices', 'English', 'Hindi']
```

```
>>> list(subjectandcode.values())
```

```
[42, 43, 41, 44, 83, 65, 101, 2]
```


Accessing Keys and Values

The keys of Dictionaries converted into list can be stored in a list variable.

```
>>> Subject=list(subjectandcode.keys())
```

```
>>>Subject
```

```
['Physics', 'Chemistry', 'Mathematics', 'Biology', 'Computer Science',  
'Informatics Practices', 'English', 'Hindi']
```

```
>>> SubjectCode=list(subjectandcode.values())
```

```
>>>SubjectCode
```

```
[42, 43, 41, 44, 83, 65, 101, 2]
```

CHARACTERISTICS OF DICTIONARY

- Dictionary is **mutable** like list. Except this there is no similarity with list. It has following attributes:
 - 1. Unordered set:** Dictionary is an unordered set of **keys:value** pairs. Its value can contain references to any type of object.
 - 2. Not a sequence:** Unlike string,, lists and tuples a **dictionary is not a sequence** because it is unordered. The sequence are indexed by a range or ordinal numbers. Hence they are ordered but dictionaries are unordered collection.
 - 3. Indexed by Keys and not Numbers:** Dictionaries are indexed by keys. According to Python, a **key can be "any non-mutable type"**. Since Strings and Numbers are non mutable it can be used as keys. Tuple if containing immutable objects (integers and strings), can be used as keys. A value in a dictionary can be of any type and type can be mixed within one dictionary.

CHARACTERISTICS OF DICTIONARY

- 4. **Keys must be unique:** Each of the **keys** within a dictionary must be **unique**. Since keys are used to identify values in a dictionary, there can not be duplicate keys in a dictionary. However **two unique keys have same value**.
- 5. **Mutable:** Like list, dictionaries are also mutable. We can change the value of a certain key “in place” using the assignment statement as per following syntax

`<dictionary>[<key>]=<value>`

```
>>> subjectandcode["Hindi"]
```

```
2
```

```
>>> subjectandcode["Hindi"]=102
```

```
>>> subjectandcode
```

```
{'Physics': 42, 'Chemistry': 43, 'Mathematics': 41, 'Biology': 44, 'Computer Science': 83, 'Informatics Practices': 65, 'English': 101, 'Hindi': 102}
```


CHARACTERISTICS OF DICTIONARY

6. **We can add new key:value pair to existing dictionary:** Using following syntax a new Key:value pair can be added to dictionary.

```
dictionary["new"]="a new pair is added"
```

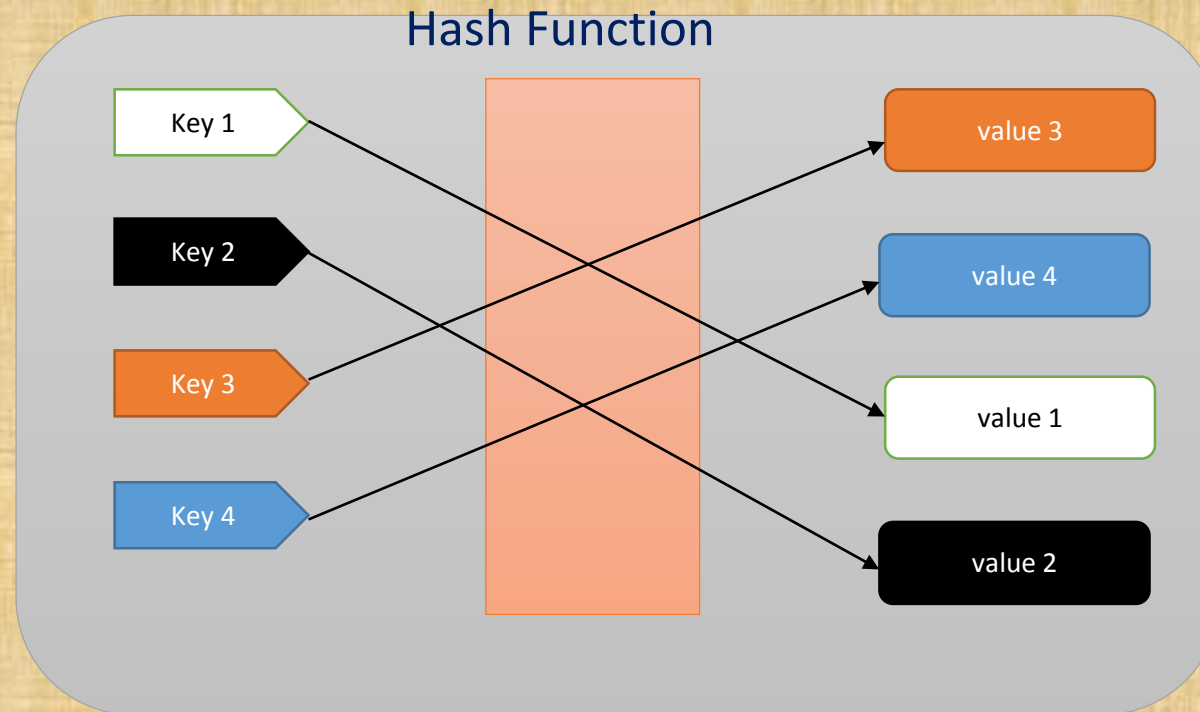
```
>>> subjectandcode["Sanskrit"]=122
```

```
>>> subjectandcode
```

```
{'Physics': 42, 'Chemistry': 43, 'Mathematics': 41, 'Biology': 44, 'Computer  
Science': 83, 'Informatics Practices': 65, 'English': 101, 'Hindi': 102, 'Sanskrit':  
122}
```

CHARACTERISTICS OF DICTIONARY

7. Internally Dictionary is stored as mapping: Internally the key:value pairs of a dictionary are associated with one another with some internal function(called hash-function-algorithm to map and link a key with a stored value). This way of linking is called mapping.



Internally Keys are mapped with values using hash function

ADDING ELEMENTS TO DICTIONARY

We can add new elements (key:value) to a dictionary using assignment as per syntax given below:

`<dictionary>[<key>]=<value>`

```
>>> Subject={"Computer":"083","Informatics Practices":"065"}
```

```
>>> Subject
```

```
{'Computer': '083', 'Informatics Practices': '065'}
```

```
>>> Subject["English"]="101"
```

```
>>> Subject
```

```
{'Computer': '083', 'Informatics Practices': '065', 'English': '101'}
```


UPDATING ELEMENTS OF DICTIONARY

We can update value of an existing element (key:value) to a dictionary using assignment as per syntax given below:

`<dictionary>[<existing key>]=<new value>`

```
>>> Subject
```

```
{'Computer': '083', 'Informatics Practices': '065', 'English': '101'}
```

```
>>> Subject["English"]="001"
```

```
>>> Subject
```

```
{'Computer': '083', 'Informatics Practices': '065', 'English': '001',}
```

Note: To add and to update the syntax is exactly same but in case adding the Key must be a new unique key and for updating the key must be an existing key and the value will be a new value by which the key value have to be changed.

DELETING ELEMENTS FROM DICTIONARY

We can delete element from a dictionary using following two ways

(a) `del <dictionary>[<key>]`

(b) `<dictionary>.pop(<key>)`

```
>>> Subject
```

```
{'Computer': '083', 'Informatics Practices': '065', 'English': '001',}
```

```
>>> del Subject["English"]
```

```
>>> Subject
```

```
{'Computer': '083', 'Informatics Practices': '065'}
```

```
>>> Subject.pop("Computer")
```

```
'083' # popped key value is returned by pop() function which is displayed
```

```
>>> Subject
```

```
{'Informatics Practices': '065', 'English': '001'}
```

```
>>> del Subject["IP"]
```

```
Traceback (most recent call last):
```

```
File "<pyshell#20>", line 1, in <module>
```

```
del Subject["IP"]
```

```
KeyError: 'IP'
```

CHECKING FOR EXISTING OF A KEY IN DICTIONARY

Usual Membership operator **in** and **not in** work with dictionary as well to check for the presence / absence of a key in the dictionary.

```
>>> emp={'age':25,"Salary":10000,"name":"sanjay"}
```

```
>>> "age" in emp
```

```
True
```

```
>>> "name" in emp
```

```
True
```

```
>>> "basic" in emp
```

```
False
```

```
>>> "basic" not in emp
```

```
True
```

But if we want to check whether a value is present in a dictionary (*called reverse lookup*) we need to write proper code.

DICTIONARY FUNCTIONS AND METHODS

SN	METHODS	PURPOSE	SYNTAX AND EXAMPLE
01.	len()	Returns length of dictionary	len<dictionary> Example >>> employee={"Name":"B Kumar","Salary":10000,"Age":45} >>> employee {'Name': 'B Kumar', 'Salary': 10000, 'Age': 45} >>> len(employee) 3
02.	clear()	Removes all elements of a list	<dictionary>.clear() >>> employee={"Name":"B Kumar","Salary":10000,"Age":45} >>> employee {'Name': 'B Kumar', 'Salary': 10000, 'Age': 45} >>> employee.clear() >>> employee {}

clear() function removes all the elements of the dictionary but the dictionary object still remains in memory.

del command used with dictionary delete dictionary object also,

DICTIONARY FUNCTIONS AND METHODS

SN	METHODS	PURPOSE	SYNTAX AND EXAMPLE
03.	get()	We can get the item with the given key similar to dictionary[key]	<dictionary>.get() Example >>> Subject={"Computer":"083","Informatics Practices":"065"} >>> Subject.get("Computer") '083'
04.	items()	Returns all of the items in the dictionary as a sequence	<dictionary>.items() >>> Subject={"Computer":"083","Informatics Practices":"065"} >>>subject >>> Subject {'Computer': '083', 'Informatics Practices': '065'} >>> Mylist=Subject.items() >>> Mylist dict_items([('Computer', '083'), ('Informatics Practices', '065')])

DICTIONARY FUNCTIONS AND METHODS

SN	METHODS	PURPOSE	SYNTAX AND EXAMPLE
05.	keys()	This method returns all the keys in the dictionary as a sequence (in the form of list)	<dictionary>.keys() Example >>> Subject={"Computer":'083',"IP":'065',"Math":'041'} >>> Subject {'Computer': '083', 'IP': '065', 'Math': '041'} >>> Subject.keys() dict_keys(['Computer', 'IP', 'Math'])
06.	values()	This method returns all the values in the dictionary as a sequence (in the form of list)	>>> Subject={"Computer":'083',"IP":'065',"Math":'041'} >>> Subject {'Computer': '083', 'IP': '065', 'Math': '041'} >>> Subject.values() dict_values(['083', '065', '041'])

DICTIONARY FUNCTIONS AND METHODS

SN	METHODS	PURPOSE	SYNTAX AND EXAMPLE
07.	update()	Merges {key:value} pairs from the new dictionary into the existing dictionary, adding or replacing as needed. The items in the new dictionary are added to the old one override any items already there with the same keys.	dictionary_to_be_updated.update(dictionary_which_to_be_updated) >>> Subject={"Computer":'083',"IP":'065',"Math":'041'} >>> Subject {'Computer': '083', 'IP': '065', 'Math': '041'} >>> Science={"Physics":'042',"Chemistry":'043',"Math":'041'} >>> Science {'Physics': '042', 'Chemistry': '043', 'Math': '041'} >>> Subject.update(Science) >>> Subject {'Computer': '083', 'IP': '065', 'Math': '041', 'Physics': '042', 'Chemistry': '043'}